5. 제약 조건 생성 모듈 핵심 분석

개요

`constraint_generation` 모듈은 Circom 컴파일러의 핵심 부분으로, 파싱된 AST와 타입 분석이 완료된 프로그램을 실제 실행 가능한 회로로 변환하는 역할을 담당합니다. 이 모듈은 템플릿 인스턴스화, R1CS 제약 조건 생성, 그리고 최종 증명 생성을 위한 VCP(Very Concrete Program) 생성까지의 전체 과정을 처리합니다.

5.1 회로 실행 및 인스턴스화 과정 분석

`build_circuit` 함수의 전체 실행 플로우

`build_circuit` 함수는 `constraint_generation/src/lib.rs` 에 정의된 메인 진입점으로, 전체 제약 조건 생성 과정을 조율합니다.

```
pub fn build_circuit(program: ProgramArchive, config: BuildConfig) -> BuildRes

let files = program.file_library.clone();

let flags = FlagsExecution{

verbose: config.flag_verbose,

inspect: config.inspect_constraints,

};

let (exe, warnings) = instantiation(&program, flags, &config.prime).map_err(|r|

Report::print_reports(&r, &files);

})?;
```

```
Report::print_reports(&warnings, &files);
let (mut dag, mut vcp, warnings) = export(exe, program, flags).map_err(|r| {
Report::print_reports(&r, &files);
})?;
if config.inspect_constraints {
Report::print_reports(&warnings, &files);
}
if config.flag_f {
sync_dag_and_vcp(&mut vcp, &mut dag);
Result::Ok((Box::new(dag), vcp))
} else {
let list = simplification_process(&mut vcp, dag, &config);
Result::Ok((Box::new(list), vcp))
}
}
```

단계별 실행 플로우:

1. **초기화 단계**

- `BuildConfig` 에서 실행 플래그 추출
- 파일 라이브러리 복사 (에러 리포팅용)
- `FlagsExecution` 구조체 생성

2. **인스턴스화 단계** (`instantiation` 함수 호출)

- 템플릿들을 실제 컴포넌트 인스턴스로 변환

- 메인 컴포넌트부터 시작하여 재귀적으로 하위 컴포넌트 생성
- `ExecutedProgram` 구조체 생성

3. **내보내기 단계** (`export` 함수 호출)

- `ExecutedProgram` 을 DAG와 VCP로 변환
- 제약 조건 그래프 구축
- 최적화를 위한 데이터 구조 준비

4. **최적화 단계**

- `flag_f` 에 따라 DAG 직접 반환 또는 단순화 과정 수행
- `sync_dag_and_vcp` : DAG와 VCP 간 증인(witness) 동기화
- `simplification_process` : 제약 조건 최적화 및 ConstraintList 생성

`instantiation` 함수의 템플릿 인스턴스화 과정

```
fn instantiation(program: &ProgramArchive, flags: FlagsExecution, prime: &Strin
let execution_result = execute::constraint_execution(&program, flags, prime);
match execution_result {
Ok((program_exe, warnings)) => {
let no_nodes = program_exe.number_of_nodes();
let success = Colour::Green.paint("template instances");
let nodes_created = format!("{}: {}", success, no_nodes);
println!("{}", &nodes_created);
InstantiationResponse::Ok((program_exe,warnings))
}
Err(reports) => InstantiationResponse::Err(reports),
```

```
}
}
```

인스턴스화 과정의 핵심:

1. **`constraint_execution` 함수 호출**

- `execute.rs` 의 메인 실행 엔진
- 메인 컴포넌트 표현식 분석 및 실행
- 재귀적 템플릿 인스턴스화

2. **실행 환경 설정**

```
let mut runtime_information = RuntimeInformation::new(*main_file_id, program_a
runtime_information.public_inputs = program_archive.get_public_inputs_main_compo
```

3. **메인 컴포넌트 실행**

```
if let Call { id, args, .. } = &program_archive.get_main_expression() {
let mut arg_values = Vec::new();
for arg_expression in args.iter() {
let f_arg = execute_expression(arg_expression, program_archive, &mut runtime_i
arg_values.push(safe_unwrap_to_arithmetic_slice(f_arg.unwrap(), line!()));
}
execute_template_call_complete(
id,
```

```
arg_values,
HashMap::new(),
program_archive,
&mut runtime_information,
flags,
)
}
```

ExecutedProgram 구조체의 역할 및 데이터 구조

ExecutedProgram 은 실행된 회로의 모든 정보를 담는 핵심 데이터 구조입니다:

```
#[derive(Default)]
pub struct ExecutedProgram {
pub model: Vec<ExecutedTemplate>, // 실행된 템플릿 인스턴스들
pub model_pretemplates: Vec<PreExecutedTemplate>, // 사전 실행된 템플릿들
pub model_buses: Vec<ExecutedBus>, // 실행된 버스들
pub template_to_nodes: HashMap<String, Vec<NodePointer>>, // 템플릿명 → 노드 포인
pub bus_to_nodes: HashMap<String, Vec<NodePointer>>, // 버스명 → 노드 포인터 매핑
pub prime: String, // 사용되는 소수 필드
}
```

주요 메서드들:

1. **노드 식별 및 관리**

```
pub fn identify_node(&self, name: &str, context: &ParameterContext, tag_contex
```

- 동일한 매개변수를 가진 템플릿 인스턴스 중복 방지
- 컨텍스트 기반 노드 식별

2. **노드 추가**

```
pub fn add_node_to_scheme(&mut self, mut node: ExecutedTemplate, analysis: A
```

- 새로운 템플릿 인스턴스를 스키마에 추가
- 분석 결과를 바탕으로 코드 정리 (`apply_unused` , `apply_computed`)
- 중복 인스턴스 검사 및 재사용

3. **내보내기 과정**

```
pub fn export(mut self, mut program: ProgramArchive, flags: FlagsExecution)
```

- DAG 구축: 각 실행된 템플릿을 DAG에 삽입
- VCP 생성: 증명 생성을 위한 최종 데이터 구조 생성
- 통계 정보 수집: 신호 수, 컴포넌트 수 등

내보내기 과정의 세부 단계:

1. **DAG 구축**

```
for exe in &mut self.model {
exe.insert_in_dag(&mut dag, &self.model_buses);
```

```
}
```

2. **버스 정보 구축**

```
let mut wrapped_buses_table = vec![None; self.model_buses.len()];
let mut index = 0;
for exe_bus in &self.model_buses{
exe_bus.build_bus_info(index, &mut wrapped_buses_table, &self.model_buses);
index += 1;
}
```

3. **템플릿 인스턴스 내보내기**

```
for exe in self.model {
let tmp_instance = exe.export_to_circuit(&mut temp_instances, &buses_table);
has_extern_c |= tmp_instance.is_extern_c;
temp_instances.push(tmp_instance);
}
```

4. **VCP 구성**

```
let config = VCPConfig {
stats: dag_stats,
main_id: dag.main_id(),
file_library: std::mem::take(&mut program.file_library),
```

```
templates: temp_instances,
templates_in_mixed: mixed,
program,
prime: self.prime,
buses: buses_table,
has_extern_c
};
let vcp = VCP::new(config);
```

실행 환경 (RuntimeInformation)

실행 과정에서 사용되는 런타임 정보를 관리하는 구조체:

```
struct RuntimeInformation {
pub block_type: BlockType, // Known/Unknown 블록 타입
pub conditions_state: Vec<(usize, bool)>, // 조건문 상태 스택
pub unknown_counter: usize, // Unknown 블록 카운터
pub analysis: Analysis, // 코드 분석 정보
pub public_inputs: Vec<String>, // 공개 입력 신호들
pub constants: UsefulConstants, // 상수 관리
pub call_trace: Vec<String>, // 호출 추적 (디버깅용)
pub current_file: FileID, // 현재 처리 중인 파일
pub runtime_errors: ReportCollection, // 런타임 에러 수집
pub environment: ExecutionEnvironment, // 실행 환경 (변수, 신호 등)
```

```
pub exec_program: ExecutedProgram, // 실행된 프로그램
pub anonymous_components: AnonymousComponentsInfo, // 익명 컴포넌트 정보
}
```

이 구조체는 템플릿 실행 과정에서 필요한 모든 컨텍스트 정보를 유지하며, 에러 처리, 변수 관리, 제약 조건 생성 등의 모든 과정에서 활용됩니다.

요구사항 충족 확인

- **Requirements 4.1**: `build_circuit` 함수의 전체 실행 플로우를 단계별로 상세히 설명
- **Requirements 4.2**: `instantiation` 함수의 템플릿 인스턴스화 과정을 `constraint_execution` 호출부터 `ExecutedProgram` 생성까지 분석
- **ExecutedProgram 구조체**: 역할, 데이터 구조, 주요 메서드들을 실제 코드와 함께 설명

이 분석을 통해 Circom 컴파일러가 어떻게 추상적인 회로 정의를 구체적인 실행 가능한 인스턴스로 변환하는지 이해할 수 있습니다.

5.2 R1CS 제약 조건 생성 과정 분석

`constraint_execution` 함수의 제약 조건 생성 알고리즘

`constraint_execution` 함수는 `execute.rs` 에서 정의되며, 실제 Circom 코드를 실행하면서 R1CS 제약 조건을 생성하는 핵심 함수입니다.

```
pub fn constraint_execution(
program_archive: &ProgramArchive,
flags: FlagsExecution,
prime: &String,
) -> Result<(ExecutedProgram, ReportCollection), ReportCollection> {
```

```
let main_file_id = program_archive.get_file_id_main();
let mut runtime_information = RuntimeInformation::new(*main_file_id, program_a
runtime_information.public_inputs = program_archive.get_public_inputs_main_compo
let folded_value_result =
if let Call { id, args, .. } = &program_archive.get_main_expression() {
let mut arg_values = Vec::new();
for arg_expression in args.iter() {
let f_arg = execute_expression(arg_expression, program_archive, &mut runtime_i
arg_values.push(safe_unwrap_to_arithmetic_slice(f_arg.unwrap(), line!()));
}
execute_template_call_complete(
id,
arg_values,
HashMap::new(),
program_archive,
&mut runtime_information,
flags,
)
} else {
unreachable!("The main expression should be a call.");
};
match folded_value_result {
Result::Err(_) => Result::Err(runtime_information.runtime_errors),
```

```
Result::Ok(folded_value) => {
debug_assert!(FoldedValue::valid_node_pointer(&folded_value));
Result::Ok((runtime_information.exec_program, runtime_information.runtime_errors
}
}
}
```

제약 조건 생성의 핵심 과정:

1. **런타임 환경 초기화**
   - `RuntimeInformation` 구조체로 실행 컨텍스트 관리
   - 공개 입력 신호 설정
   - 소수 필드 및 상수 관리 초기화
2. **메인 컴포넌트 실행**
   - 메인 컴포넌트 호출 표현식 분석
   - 매개변수 평가 및 템플릿 인스턴스화
   - `execute_template_call_complete` 호출
3. **제약 조건 생성 트리거**
   - 신호 할당 연산자 (`<==` , `==>`) 처리
   - 등식 제약 조건 (`===`) 처리
   - 산술 표현식을 R1CS 형태로 변환

산술 표현식을 R1CS 형태로 변환하는 과정

ArithmeticExpression 데이터 구조

Circom의 모든 산술 표현식은 `circom_algebra` 모듈의 `ArithmeticExpression` 열거형으로 표현됩니다:

```
pub enum ArithmeticExpression<C>
where
C: Hash + Eq,
{
Number {
value: BigInt,
},
Signal {
symbol: C,
},
Linear {
// c1*s1 + .. + cn*sn + C 형태의 선형 표현식
coefficients: HashMap<C, BigInt>,
},
Quadratic {
// a*b + c 형태의 이차 표현식 (R1CS 호환)
// a, b, c는 모두 선형 표현식
a: HashMap<C, BigInt>,
b: HashMap<C, BigInt>,
c: HashMap<C, BigInt>,
},
```

```
NonQuadratic, // R1CS로 표현 불가능한 고차 표현식
}
```

R1CS 변환 과정

1. **제약 조건 신호 할당 (<==) 처리**

```
// execute.rs의 Substitution 처리 부분
if *op == AssignOp::AssignConstraintSignal {
if  value_right.is_nonquadratic() {
let  err = Result::Err(ExecutionError::NonQuadraticConstraint);
// 에러 처리
} else {
let  p = runtime.constants.get_p().clone();
let  symbol = signal_left;
let  expr = AExpr::sub(&symbol, &value_right, &p);
let  ctr = AExpr::transform_expression_to_constraint_form(expr, &p).unwrap();
node.add_constraint(ctr);
}
}
```

2. **`transform_expression_to_constraint_form` 함수**

```
pub  fn  transform_expression_to_constraint_form(
arithmetic_expression: ArithmeticExpression<C>,
```

```
field: &BigInt,

) -> Option<Constraint<C>> {

use ArithmeticExpression::*;

let  mut  a = HashMap::new();

let  mut  b = HashMap::new();

let  mut  c = HashMap::new();

match  arithmetic_expression {

NonQuadratic => return Option::None,

Quadratic { a: old_a, b: old_b, c: old_c } => {

a = old_a;

b = old_b;

c = old_c;

}

Number { value } => {

c.insert(ArithmeticExpression::constant_coefficient(), value);

}

Signal { symbol } => {

c.insert(symbol, BigInt::from(1));

}

Linear { coefficients } => {

c = coefficients;

}

}
```

```
// c 계수에 -1을 곱하여 A*B - C = 0 형태로 변환
ArithmeticExpression::multiply_coefficients_by_constant(&BigInt::from(-1), &mut
Option::Some(Constraint::new(a, b, c))
}
```

3. **R1CS 제약 조건 구조**

```
pub struct Constraint<C> {
pub a: HashMap<C, BigInt>, // A 벡터 (신호 → 계수 매핑)
pub b: HashMap<C, BigInt>, // B 벡터
pub c: HashMap<C, BigInt>, // C 벡터
}
// 제약 조건: (A · witness) * (B · witness) = (C · witness)
```

산술 연산별 R1CS 변환 예시

1. **덧셈 연산 (a + b)**

```
pub fn add(left: &ArithmeticExpression<C>, right: &ArithmeticExpression<C>, fi
match (left, right) {
(Signal { symbol: s1 }, Signal { symbol: s2 }) => {
let mut coefficients = HashMap::new();
coefficients.insert(s1.clone(), BigInt::from(1));
coefficients.insert(s2.clone(), BigInt::from(1));
```

```
Linear { coefficients }

}

// 다른 경우들...

}

}
```

2. **곱셈 연산 (a * b)**

```
pub fn mul(left: &ArithmeticExpression<C>, right: &ArithmeticExpression<C>, fi

match (left, right) {

(Signal { symbol: s1 }, Signal { symbol: s2 }) => {

let mut a = HashMap::new();

let mut b = HashMap::new();

let mut c = HashMap::new();

a.insert(s1.clone(), BigInt::from(1));

b.insert(s2.clone(), BigInt::from(1));

Quadratic { a, b, c }

}

// 다른 경우들...

}

}
```

제약 조건 검증 및 일관성 체크 메커니즘

1. 이차식 검증 (Quadratic Constraint Validation)

```
// execute.rs의 제약 조건 처리 부분
if value_right.is_nonquadratic() {
let err = Result::Err(ExecutionError::NonQuadraticConstraint);
treat_result_with_execution_error(
err,
meta,
&mut runtime.runtime_errors,
&runtime.call_trace,
)?;
}
```

2. 등식 제약 조건 처리 (===)

```
// ConstraintEquality 문 처리
ConstraintEquality { meta, lhe, rhe, .. } => {
let f_left = execute_expression(lhe, program_archive, runtime, flags)?;
let f_right = execute_expression(rhe, program_archive, runtime, flags)?;
let possible_non_quadratic = AExpr::sub(&value_left, &value_right, &runtime.con
if possible_non_quadratic.is_nonquadratic() {
treat_result_with_execution_error(
Result::Err(ExecutionError::NonQuadraticConstraint),
meta,
```

```
&mut runtime.runtime_errors,
&runtime.call_trace,
)?;
}
let constraint_expression = AExpr::transform_expression_to_constraint_form(
possible_non_quadratic,
runtime.constants.get_p(),
).unwrap();
if let Option::Some(node) = actual_node {
node.add_constraint(constraint_expression);
}
}
```

3. 제약 조건 저장 및 관리

`ExecutedTemplate` 구조체에서 제약 조건을 관리합니다:

```
pub struct ExecutedTemplate {
pub constraints: Vec<Constraint>, // 생성된 제약 조건들
// 다른 필드들...
}

impl ExecutedTemplate {
pub fn add_constraint(&mut self, constraint: Constraint) {
```

```
self.constraints.push(constraint);
}
fn build_constraints(&self, dag: &mut DAG) {
for c in &self.constraints {
let correspondence = dag.get_main().unwrap().correspondence();
let cc = Constraint::apply_correspondence(c, correspondence);
dag.add_constraint(cc);
}
}
}
```

4. 신호 대응 관계 (Signal Correspondence) 적용

DAG에 제약 조건을 추가할 때 신호 이름을 전역 인덱스로 매핑:

```
impl Constraint<C> {
pub fn apply_correspondence(constraint: &Constraint<C>, correspondence: &HashM
let mut new_a = HashMap::new();
let mut new_b = HashMap::new();
let mut new_c = HashMap::new();
for (signal, coeff) in &constraint.a {
if let Some(new_signal) = correspondence.get(signal) {
new_a.insert(new_signal.clone(), coeff.clone());
}
```

```
}
// b, c에 대해서도 동일한 처리
Constraint { a: new_a, b: new_b, c: new_c }
}
}
```

5. 제약 조건 압축 및 최적화

`constraint_storage` 모듈에서 제약 조건을 효율적으로 저장:

```
pub struct ConstraintStorage {
field_tracker: FieldTracker, // 계수 값 추적
constraints: Vec<CompressedConstraint>, // 압축된 제약 조건들
}

impl ConstraintStorage {
pub fn add_constraint(&mut self, constraint: C) -> ConstraintID {
let id = self.constraints.len();
let compressed = logic::code_constraint(constraint, &mut self.field_tracker);
self.constraints.push(compressed);
id
}
}
```

제약 조건 생성 과정의 최적화

1. **상수 전파 (Constant Propagation)**

- 컴파일 타임에 알려진 값들을 미리 계산
- `constant_tracking` 모듈과 연동

2. **중복 제약 조건 제거**

- 동일한 제약 조건 중복 생성 방지
- 해시맵 기반 중복 검사

3. **신호 인덱싱 최적화**

- 전역 신호 인덱스 할당
- 메모리 효율적인 신호 참조

요구사항 충족 확인

- **Requirements 4.1**: `constraint_execution` 함수의 제약 조건 생성 알고리즘을 실제 코드와 함께 상세 분석
- **Requirements 4.3**: 산술 표현식을 R1CS 형태로 변환하는 `transform_expression_to_constraint_form` 과정을 단계별로 설명
- **제약 조건 검증**: 이차식 검증, 등식 제약 조건 처리, 신호 대응 관계 적용 등의 일관성 체크 메커니즘 문서화

이 분석을 통해 Circom이 어떻게 고수준 회로 정의를 저수준 R1CS 제약 조건으로 변환하는지 이해할 수 있습니다.## 5.3 VCP

(Very Concrete Program) 생성 과정 분석

`export` 함수의 VCP 생성 과정

VCP(Very Concrete Program)는 증명 생성을 위한 최종 데이터 구조로, `ExecutedProgram` 에서 DAG와 함께 생성됩니다.

```
pub fn export(mut self, mut program: ProgramArchive, flags: FlagsExecution)
let mut warnings = vec![];
let mut dag = DAG::new(&self.prime);
let mut temp_instances = Vec::with_capacity(self.model.len());
let mut mixed_instances = vec![false; self.model.len()];

// 1. 혼합 인스턴스 분석
for exe in &self.model {
let mixed = templates_in_mixed_arrays(exe, self.model.len());
mixed_instances = merge_mixed(mixed_instances, mixed);
}

// 2. DAG 구축
for exe in &mut self.model {
exe.insert_in_dag(&mut dag, &self.model_buses);
}

// 3. 버스 정보 구축
let mut wrapped_buses_table = vec![None; self.model_buses.len()];
let mut index = 0;
for exe_bus in &self.model_buses{
exe_bus.build_bus_info(index, &mut wrapped_buses_table, &self.model_buses);
```

```
index += 1;
}

// 4. 템플릿 인스턴스 내보내기
let mut has_extern_c = false;
for exe in self.model {
let tmp_instance = exe.export_to_circuit(&mut temp_instances, &buses_table);
has_extern_c |= tmp_instance.is_extern_c;
temp_instances.push(tmp_instance);
}

// 5. DAG 정리 및 분석
temp_instances[dag.main_id()].is_not_parallel_component = true;
dag.clean_constraints();
if flags.inspect{
let mut w = dag.constraint_analysis()?;
warnings.append(&mut w);
}

// 6. VCP 구성
let dag_stats = produce_dags_stats(&dag);
crate::compute_constants::manage_functions(&mut program, flags, &self.prime)?;
crate::compute_constants::compute_vct(&mut temp_instances, &program, flags, &se
```

```
let  config = VCPConfig {
stats: dag_stats,
main_id: dag.main_id(),
file_library: std::mem::take(&mut  program.file_library),
templates: temp_instances,
templates_in_mixed: mixed,
program,
prime: self.prime,
buses: buses_table,
has_extern_c
};
let  vcp = VCP::new(config);
Result::Ok((dag, vcp, warnings))
}
```

VCP 생성 과정의 단계별 분석:

1. **혼합 인스턴스 분석 (Mixed Instance Analysis)**

```
fn  templates_in_mixed_arrays(exec_tmp: &ExecutedTemplate, no_templates: usize)
let  solution = apply_pos_to_connexions(&exec_tmp.connexions);
let  mut  mixed = vec![false; no_templates];
for  cnn  in &exec_tmp.connexions {
let  data = &cnn.inspect;
```

```
let  pos_value = solution.get(&data.name).unwrap();

mixed[data.goes_to] = mixed[data.goes_to] || matches!(pos_value, T);

}

mixed

}
```

- 템플릿 배열에서 혼합된 인스턴스 식별
- 동적 컴포넌트 할당 패턴 분석

2. **DAG 구축 과정**

```
fn  insert_in_dag(&mut  self, dag: &mut DAG, buses_info : &Vec<ExecutedBus>) {

// 노드 추가

dag.add_node(

self.report_name.clone(),

parameters,

self.is_parallel,

self.is_custom_gate

);

// 신호, 연결, 제약 조건 구축

self.build_wires(dag, buses_info);

self.build_ordered_signals(dag, buses_info);

self.build_connexions(dag);

self.build_constraints(dag);
```

```
}
```

3. **템플릿 인스턴스 변환**

```
pub fn export_to_circuit(self, instances: &mut [TemplateInstance], buses_info
let config = TemplateConfig {
header: format!("{}_{}", self.template_name, instances.len()),
clusters: build_clusters(&self, instances),
triggers: build_triggers(instances, self.connexions),
components: build_components(self.components),
arguments: build_arguments(self.parameter_instances),
// 기타 설정...
};
TemplateInstance::new(config)
}
```

DAG와 VCP 간의 동기화 메커니즘 (`sync_dag_and_vcp`)

```
fn sync_dag_and_vcp(vcp: &mut VCP, dag: &mut DAG) {
let witness = Rc::new(DAG::produce_witness(dag));
VCP::add_witness_list(vcp, Rc::clone(&witness));
}
```

증인(Witness) 생성 과정:

1. **DAG 기반 증인 생성**

```
pub fn produce_witness(&self) -> Vec<usize> {
let mut witness = vec![0]; // 첫 번째 원소는 항상 1 (상수)
let tree = Tree::new(self);
produce_tree_witness(&tree, &mut witness);
Vec::shrink_to_fit(&mut witness);
witness
}
fn produce_tree_witness(tree: &Tree, witness: &mut Vec<usize>) {
// 현재 노드의 모든 신호를 증인에 추가
for signal in &tree.signals {
Vec::push(witness, *signal);
}
// 하위 트리 재귀적 처리
for edge in Tree::get_edges(tree) {
let subtree = Tree::go_to_subtree(tree, edge);
produce_tree_witness(&subtree, witness);
}
}
```

2. **신호 순서 보장**

- DAG의 위상 정렬 순서에 따른 신호 배치
- 입력 → 출력 → 중간 신호 순서 유지

- 하위 컴포넌트 신호들의 계층적 배치

증명 생성을 위한 최종 데이터 구조 변환 과정

VCP 구조체의 핵심 구성 요소:

```
pub struct VCP {
pub stats: Stats, // 회로 통계 정보
pub main_id: usize, // 메인 템플릿 ID
pub functions: Vec<VCF>, // 함수 정의들
pub file_library: FileLibrary, // 파일 라이브러리
pub witness_list: Rc<Vec<usize>>, // 증인 리스트
pub templates: Vec<TemplateInstance>, // 템플릿 인스턴스들
pub quick_knowledge: HashMap<String, VCT>, // 타입 정보 캐시
pub templates_in_mixed: Vec<usize>, // 혼합 배열의 템플릿들
pub prime: String, // 소수 필드
pub buses: Vec<BusInstance>, // 버스 인스턴스들
pub has_extern_c: bool // C 외부 함수 사용 여부
}
```

VCP 전처리 과정 (`run_preprocessing`):

```
pub fn run_preprocessing(vcp: &mut VCP, program_archive: ProgramArchive) {
let mut state = build_function_knowledge(program_archive);
// 1. VCF (Very Concrete Function) 생성
```

```
produce_vcf(vcp, &mut state);
// 2. 회로 연결 (Linking)
link_circuit(vcp, &mut state);
// 3. 지식 베이스 구축
vcp.quick_knowledge = state.quick_knowledge;
vcp.functions = state.vcf_collector;
// 4. 컴포넌트 전처리
component_preprocess::rm_component_ci(vcp);
// 5. 문법 설탕 정리
sugar_cleaner::clean_sugar(vcp);
}
```

통계 정보 생성 (`produce_dags_stats`):

```
fn produce_dags_stats(dag: &DAG) -> Stats {
let mut all_created_cmp = vec![0; dag.number_of_nodes()];
let mut all_needed_subcomponents_indexes = vec![0; dag.number_of_nodes()];
let mut all_signals = vec![0; dag.number_of_nodes()];
let mut all_io = vec![0; dag.number_of_nodes()];
for (index, node) in dag.nodes.iter().enumerate() {
all_signals[index] += node.number_of_inputs();
all_signals[index] += node.number_of_outputs();
all_signals[index] += node.number_of_intermediates();
all_io[index] += node.number_of_inputs();
```

```
all_io[index] += node.number_of_outputs();

all_created_cmp[index] += 1;

all_needed_subcomponents_indexes[index] += node.number_of_subcomponents_indexes(

// 하위 컴포넌트 통계 누적

for  c  in  dag.get_edges(index).unwrap() {

all_created_cmp[index] += all_created_cmp[c.get_goes_to()];

all_needed_subcomponents_indexes[index] += all_needed_subcomponents_indexes[c.ge

all_signals[index] += all_signals[c.get_goes_to()];

all_io[index] += all_io[c.get_goes_to()];

}

}



Stats {

all_signals: all_signals.pop().unwrap(),

io_signals: all_io.pop().unwrap(),

all_created_components: all_created_cmp.pop().unwrap(),

all_needed_subcomponents_indexes: all_needed_subcomponents_indexes.pop().unwrap(

}

}
```

VCP의 최종 활용

1. **증명 생성 라이브러리 연동**

- snarkjs: JavaScript 환경에서의 증명 생성
- rapidSnark: C++ 환경에서의 고성능 증명 생성
- 생성된 .wasm 파일과 .r1cs 파일 활용

2. **코드 생성**

```
// C++ 코드 생성
pub fn produce_c_code(vcp: &VCP, config: &CompilerConfig) -> Result<(), ()>
// WebAssembly 코드 생성
pub fn produce_wasm_code(vcp: &VCP, config: &CompilerConfig) -> Result<(), ()>
```

3. **심볼 파일 생성**

- 신호 이름과 인덱스 매핑 정보
- 디버깅 및 검증을 위한 메타데이터

최적화 및 단순화 과정

VCP 생성 후 추가적인 최적화가 수행됩니다:

```
fn simplification_process(vcp: &mut VCP, dag: DAG, config: &BuildConfig) -> Con
let flags = SimplificationFlags {
flag_s: config.flag_s,
parallel_flag: config.flag_p,
port_substitution: config.flag_json_sub,
json_substitutions: config.json_substitutions.clone(),
no_rounds: config.no_rounds,
```

```
flag_old_heuristics: config.flag_old_heuristics,

prime : config.prime.clone(),

};

let  list = DAG::map_to_list(dag, flags);

VCP::add_witness_list(vcp, Rc::new(list.get_witness_as_vec()));

list

}
```

요구사항 충족 확인

- **Requirements 4.2**: `export` 함수의 VCP 생성 과정을 단계별로 상세 분석
- **Requirements 4.3**: `sync_dag_and_vcp` 함수를 통한 DAG와 VCP 간 동기화 메커니즘 설명
- **증명 생성 데이터 구조**: VCP의 구조, 전처리 과정, 통계 정보 생성 등 최종 데이터 구조 변환 과정 문서화

이 분석을 통해 Circom 컴파일러가 어떻게 실행된 회로를 증명 생성에 최적화된 형태로 변환하는지 이해할 수 있습니다. VCP는 단순히 데이터 구조가 아니라, zkSNARK 증명 시스템과의 효율적인 인터페이스를 제공하는 핵심 구성 요소입니다.